

LINFO2315 : 1D Stabilizer

Design Choices

Hardware Components

In this part, we discuss the choices made for interfacing the different hardware components of the system.

Gyroscope

The component used is an MPU6050. Hence, we have access to both an accelerometer and a gyroscope. But we only consider the *gyroscope* since we are interested in measuring an angle.

The gyroscope measures only the *angular speed* in $\left[\frac{^\circ}{s}\right]$ (degrees per second) along its 3 axes. Hence, if we want to get the *angular position*, we have to integrate the speed.

The MPU6050 uses the I²C protocol for communication at 400kHz. Giving us access to its internal registers. It can be configured to work at different ranges of speed with different precisions. For this application, we stick with the $\pm 250 \left[\frac{^\circ}{s}\right]$ (the smallest range) so that we can benefit from the maximum precision. This choice is based on the assumption that the system will never rotate with high speed.

Rotary Encoder

The rotary encoder is fairly easy to interface, it sends pulses on two channels with different phases depending on the direction of its rotation. Hence, we simply use a *pulse counter* over the two channels.

For the button part of the encoder, it has a dedicated pin that will drive the current low when pressed.

Therefore, we can interface it like any other button. Except that we need to be able to distinguish long from short presses. To achieve that we don't raise interruption, we use `wait_for_falling_edge()` and `wait_for_rising_edge()` and measure the time between those two edges.

Servo Motor

The servo motor is controlled by changing the *duty cycle* of a pulse. To achieve this, we use a *pulse width modulator*, which is typically used for LED control. Given that the frequency of the pulse must be fairly low (50Hz), we can benefit from a higher precision by using more bits. For our application, we choose a 12 bits resolution in a complete arbitrary manner.

Software Components

In this part, we discuss the partitioning of the system logic into tasks. As well as the transfer of data across the different tasks.

Refer to Figure 1 for the architecture of the tasks.

Tasks Frequencies

Gyroscope: runs at high frequency (as fast as possible) in order to have a somewhat accurate measure of the current angle.

Motor Control: runs at high frequency (follows the gyroscope loop) such that the motor replicates the internal state as tightly as possible.

Threshold: runs at a moderate frequency (a few hertz) since it is interfacing with user input.

Button: aperiodic, uses asynchronous code.

UI: runs at low frequency ($\sim 1\text{Hz}$) to prevent heavy load on the cpu. Given that it requires graphical processing.

Main: runs at low frequency ($\sim 2\text{Hz}$) since it only monitor internal state for debugging purposes.

Shared State

Because of the high discrepancy of frequency among the tasks, it is easier to work with a *shared state* rather than channels. The *shared state* is protected by *atomic* operations instead of using *mutexes*.

We have chosen atomics over mutexes because we are working with a few raw values. And also because no task requires to write to multiple fields of the state. Mutexes add too much overhead for such simple and minimal modifications.

Channel

The only channel used between two tasks is the one going from the *gyroscope task* to the *motor control task*. This channel allows to “synchronize” the two tasks. In the sens, that for every loop of the gyroscope, we expect the motor control to be updated once.

Diagram

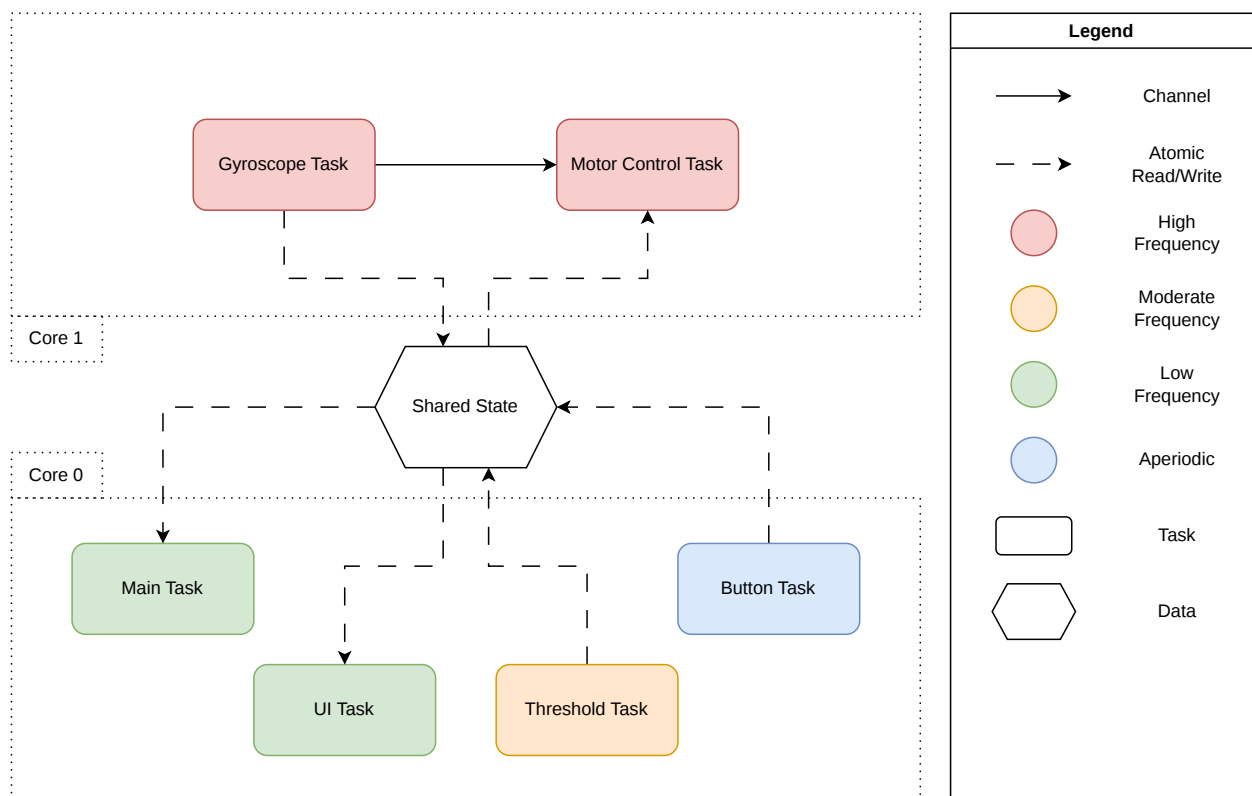


Figure 1: Dispatching of the tasks across the CPU cores and communication medium between tasks.